Activity-Based Cost & GHG Calculator: A MIS-based Add-On Application for the Production Domain

AB_Cost_GHG_Calculator_2026

Walter S.A. Schwaiger (IMW/TU Wien)

July 22, 2026

Abstract

Solving the problem of building an add-on application of the activity-based cost & GHG accounting in the production domain requires an understanding of how production systems as well as cost and GHG accounting are working, how this combined knowledge is represented in an information system and how this system is implemented in an add-on application that receives information from the ERP system (ERPS) and extends its cost & GHG accounting functionalities. In this article the ERPS's provided information is unified in a common name space (ITEM.tbl), converted into a production information system (MIS) graph and evaluated concerning the production costs (PC) and the production carbon footprint (PCF) by applying the recursive cost and GHG engine (recursion engine) over the MIS graph.

Contents

1 Loading: Libraries, ERP Data and Accounting Logic

1.1 Loading Libraries (Dependencies)

```r
library(tidyverse)
```

```r
knitr::include_graphics("SlotCar_BOM.png")
```

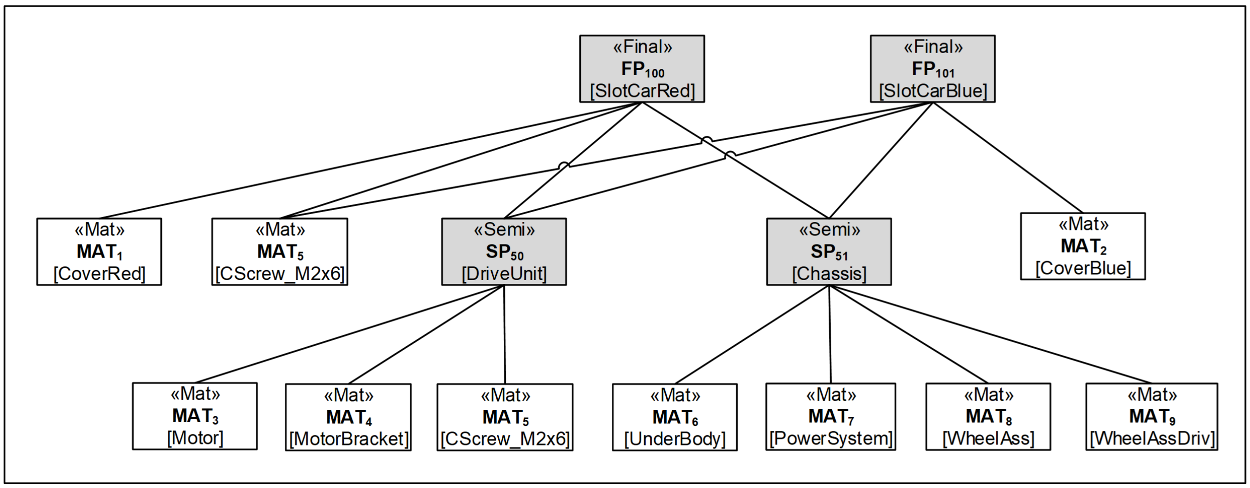


Figure 1: SlotCar's BOM

```r
knitr::include_graphics("SlotCar_Routing.png")
```

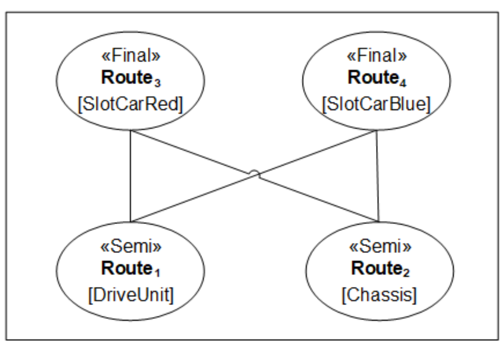


Figure 2: SlotCar's ROUTING

```r
knitr::include_graphics("SlotCar_MIS.png")
```

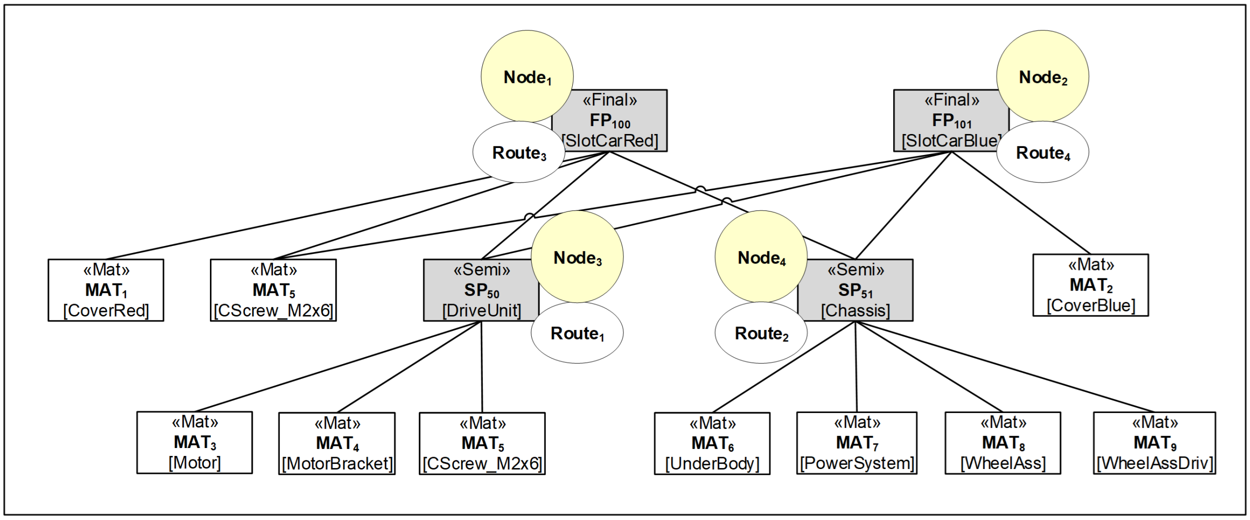


Figure 3: SlotCar's MIS

1.2 Loading Data from ERP System

```
load("SlotCar.RData")
```

```
ls()
```

```
## [1] "ACTY.erp"    "BOM.erp"     "ENERGY.erp"  "EQIP.erp"    "FINAL.erp"
## [6] "MAT.erp"     "ROUTING.erp" "SEMI.erp"
```

1.3 Loading Cost & GHG Accounting Logic

```
source("01_item_master_builder.R")
source("02_erp_mapper.R")
source("03_mis_graph_builder.R")
source("04_ais_recursion_engine.R")
source("05_ais_analyzer.R")
```

---

2 ERP-System Data Delivery: Viewing Data

```
MAT.erp
```

```
## # A tibble: 9 x 4
##   matID matName                pM   uMEL
##   <dbl> <chr>               <dbl>  <dbl>
## 1     1 CoverRed            29    0.15
## 2     2 CoverBlue           28    0.1
## 3     3 Motor               10    0.2
```

```
## 4      4 MotorMountingBracket  0.12 0.0006
## 5      5 CScrew_M2x6           0.02 0.05
## 6      6 UnderBody             1.44 0.0072
## 7      7 PowerSystem           3.26 0.06
## 8      8 WheelAssembly        13.4  0.766
## 9      9 WheelAssemblyDriven  18.2  1.1
```

```
SEMI.erp
```

```
## # A tibble: 2 x 2
##    spID spName
##   <dbl> <chr>
## 1    50 DriveUnit
## 2    51 Chassis
```

```
FINAL.erp
```

```
## # A tibble: 2 x 2
##    fpID fpName
##   <dbl> <chr>
## 1   100 SlotCarRed
## 2   101 SlotCarBlue
```

```
BOM.erp # flat BOM construction (i.e. w/o list columns)
```

```
## # A tibble: 15 x 6
##    bomID parCat parID childCat childID    rM
##    <dbl> <chr>  <dbl> <chr>      <dbl> <dbl>
##  1     1 semi      50 mat            3     1
##  2     2 semi      50 mat            4     1
##  3     3 semi      50 mat            5     1
##  4     4 semi      51 mat            6     1
##  5     5 semi      51 mat            7     1
##  6     6 semi      51 mat            8     1
##  7     7 semi      51 mat            9     1
##  8     8 final    100 mat            1     1
##  9     9 final    100 mat            5     1
## 10    10 final    100 semi          50     1
## 11    11 final    100 semi          51     1
## 12    12 final    101 mat            2     1
## 13    13 final    101 mat            5     1
## 14    14 final    101 semi          50     1
## 15    15 final    101 semi          51     1
```

```
ROUTING.erp # flat ROUTING construction  (i.e. w/o list columns)
```

```
## # A tibble: 4 x 7
##   routeID semFinCat semFinID actyID    rE eqipID  powE
##     <dbl> <chr>        <dbl>  <dbl> <dbl>  <dbl> <dbl>
## 1       1 semi            50      1   7.5      2  0.05
## 2       2 semi            51      2  51        1  0.05
```

```
## 3       3 final          100      3   1.5      1  0.05
## 4       4 final          101      3   1.5      1  0.05
```

```
ACTY.erp
```

```
## # A tibble: 3 x 3
##   actyID actyName             pA
##    <dbl> <chr>             <dbl>
## 1      1 DriveUnitAssembly     2
## 2      2 ChassisAssembly       2
## 3      3 SlotCarAssembly       2
```

```
EQIP.erp
```

```
## # A tibble: 2 x 5
##   eqipID eqipName             qE   cuEEL enyID
##    <dbl> <chr>             <dbl>   <dbl> <dbl>
## 1      1 AssemblerElectric   0.3 0.00312     1
## 2      2 AssemblerGas        0.3 0.00312     2
```

```
ENERGY.erp
```

```
## # A tibble: 2 x 4
##   enyID enyName       emE scopeCat
##   <dbl> <chr>       <dbl>    <dbl>
## 1     1 Electricity 0.069        2
## 2     2 Gas         0.069        1
```

3 Item Master: Building ITEM.tbl

3.1 Building ITEM.tbl

Building *ITEM.tbl* using *buildItemMaster()*

```
ITEM.tbl <- buildItemMaster(MAT.erp, SEMI.erp, FINAL.erp)
```

3.2 Viewing ITEM.tbl

Viewing *ITEM.tbl*

```
ITEM.tbl
```

```
## # A tibble: 13 x 5
##    itemID erpID erpCat itemName             itemCat
##     <int> <dbl> <chr>  <chr>                <chr>
##  1      1   100 final  SlotCarRed           finished
##  2      2   101 final  SlotCarBlue          finished
##  3      3    50 semi   DriveUnit            intermediate
##  4      4    51 semi   Chassis              intermediate
##  5      5     1 mat    CoverRed             material
##  6      6     2 mat    CoverBlue            material
```

```
## 7       7      3 mat    Motor                 material
## 8       8      4 mat    MotorMountingBracket material
## 9       9      5 mat    CScrew_M2x6          material
## 10     10      6 mat    UnderBody            material
## 11     11      7 mat    PowerSystem          material
## 12     12      8 mat    WheelAssembly        material
## 13     13      9 mat    WheelAssemblyDriven  material
```

---

4 Item Mapper: Mapping ERP to ITEM

4.1 Building BOM.tbl, ROUTING.tbl and MAT.tbl

Mapping *BOM.erp*, *ROUTING.erp* and *MAT.erp* to *ITEM.tbl* using *mapERPtoITEM()*

```
mapERPtoITEM(ITEM.tbl, BOM.erp, ROUTING.erp, MAT.erp)
```

4.2 Viewing BOM.tbl, ROUTING.tbl and MAT.tbl

Viewing *BOM.tbl*

```
BOM.tbl
```

```
## # A tibble: 15 x 4
##    bomID parItemID childItemID    rM
##    <dbl>     <int>       <int> <dbl>
##  1     1         3           7     1
##  2     2         3           8     1
##  3     3         3           9     1
##  4     4         4          10     1
##  5     5         4          11     1
##  6     6         4          12     1
##  7     7         4          13     1
##  8     8         1           5     1
##  9     9         1           9     1
## 10    10         1           3     1
## 11    11         1           4     1
## 12    12         2           6     1
## 13    13         2           9     1
## 14    14         2           3     1
## 15    15         2           4     1
```

Viewing *ROUTING.tbl*

```
ROUTING.tbl
```

```
## # A tibble: 4 x 6
##   routeID itemID actyID    rE eqipID  powE
##     <dbl>  <int>  <dbl> <dbl>  <dbl> <dbl>
## 1       1      3      1   7.5      2  0.05
## 2       2      4      2  51        1  0.05
## 3       3      1      3   1.5      1  0.05
## 4       4      2      3   1.5      1  0.05
```

Viewing *MAT.tbl*

```
MAT.tbl
```

```
## # A tibble: 9 x 3
##   itemID    pM   uMEL
##    <int> <dbl>  <dbl>
## 1      5 29    0.15
## 2      6 28    0.1
## 3      7 10    0.2
## 4      8  0.12 0.0006
## 5      9  0.02 0.05
## 6     10  1.44 0.0072
## 7     11  3.26 0.06
## 8     12 13.4  0.766
## 9     13 18.2  1.1
```

---

5 MIS Graph (Digital Twin) Engine: Building MIS.tbl

5.1 Building MIS.tbl

Building *MIS.tbl* using *buildMIS()*

```
MIS.tbl <- buildMIS(ITEM.tbl, ROUTING.tbl, ACTY.erp, BOM.tbl, EQIP.erp, ENERGY.erp)
MIS.tbl <- MIS.tbl |> select(nodeID, chilNodeIDs, routeID:matList)
```

5.2 Viewing MIS.tbl

Viewing *MIS.tbl*

```
MIS.tbl |> select(nodeID, chilNodeIDs, itemID, itemName, actyName, eqipList, matList)
```

```
## # A tibble: 4 x 7
##   nodeID chilNodeIDs itemID itemName    actyName          eqipList matList
##    <int> <list>       <int> <chr>       <chr>             <list>   <list>
## 1      1 <int [2]>        1 SlotCarRed  SlotCarAssembly   <tibble> <tibble>
## 2      2 <int [2]>        2 SlotCarBlue SlotCarAssembly   <tibble> <tibble>
## 3      3 <int [0]>        3 DriveUnit   DriveUnitAssembly <tibble> <tibble>
## 4      4 <int [0]>        4 Chassis     ChassisAssembly   <tibble> <tibble>
```

---

6 AIS Recursion Engine: Building AIS.tbl

6.1 Calculating AIS.tbl

Calculating *AIS.tbl* using *buildMultiItemAccData()*

```
# nodeIDs = c(1:4)
AIS.tbl <- buildMultiItemAccData(c(1:4), MIS.tbl, ITEM.tbl, MAT.tbl)
```

Table 1: PC and PCF of (semi-)finished products

nodeID	PC	PCF
1	213.6472	2.778000
2	212.6472	2.728000
3	27.4134	0.299875
4	153.7591	2.268270

6.2 Viewing AIS.tbl

Viewing *AIS.tbl*

```
AIS.tbl |> select(nodeID, PC, PCF) |>
  knitr::kable(caption = "PC and PCF of (semi-)finished products")
```

```
AIS.tbl
```

```
## # A tibble: 4 x 5
##   nodeID    PC   PCF nodeCompsList     nodeTotalsList
##    <int> <dbl> <dbl> <list>            <list>
## 1      1 214.   2.78  <tibble [12 x 6]> <tibble [3 x 3]>
## 2      2 213.   2.73  <tibble [12 x 6]> <tibble [3 x 3]>
## 3      3  27.4 0.300 <tibble [4 x 6]>  <tibble [1 x 3]>
## 4      4 154.   2.27  <tibble [4 x 6]>  <tibble [1 x 3]>
```

```
AIS.tbl |> select(nodeID, nodeTotalsList) |> knitr::kable()
```

nodeID	nodeTotalsList
1	1.000000, 3.000000, 4.000000, 32.474680, 27.413400, 153.759120, 0.209855, 0.299875, 2.268270
2	2.000000, 3.000000, 4.000000, 31.474680, 27.413400, 153.759120, 0.159855, 0.299875, 2.268270
3	3.000000, 27.413400, 0.299875
4	4.00000, 153.75912, 2.26827

```
AIS.tbl$nodeTotalsList[[1]]
```

```
## # A tibble: 3 x 3
##   nodeID nodePC nodePCF
##    <int>  <dbl>   <dbl>
## 1      1   32.5   0.210
## 2      3   27.4   0.300
## 3      4  154.    2.27
```

---

7 AIS Data Analysis: Comparison of Cost & GHG Structures

7.1 Building Comparison.tbl

Building *Comparison.tbl* using *buildMultiItemCatShareData()*

```
# nodeID = c(1,2)
Comparison.tbl <- buildMultiItemCatShareData(c(1,2), AIS.tbl)
```

Table 2: Comparative analysis of PC and PCF structures

nodeID	category	share	percent
1	direct	Cost	0.3531991
1	indirect	Cost	0.6468009
1	1	Emission	0.0093143
1	2	Emission	0.0651998
1	3	Emission	0.9254860
2	direct	Cost	0.3501574
2	indirect	Cost	0.6498426
2	1	Emission	0.0094850
2	2	Emission	0.0663948
2	3	Emission	0.9241202

7.2 Viewing Comparison.tbl

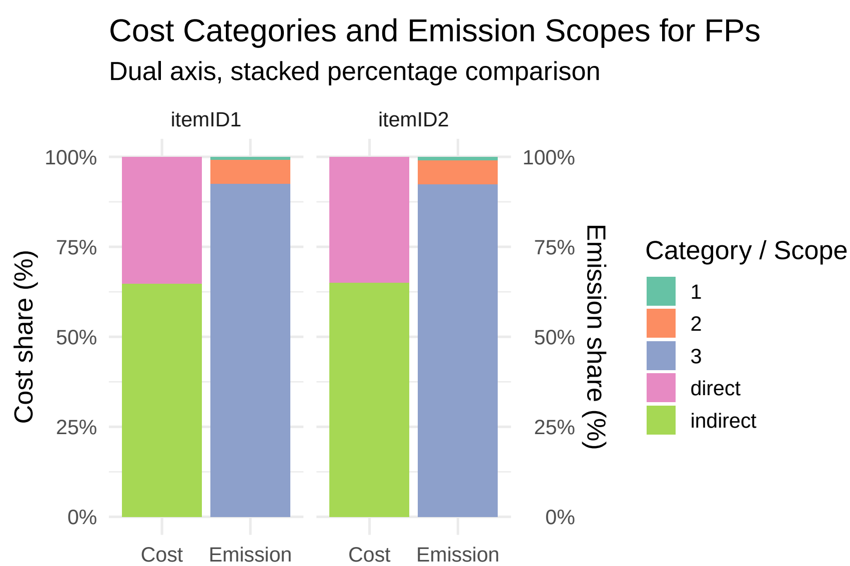
Viewing *Comparison.tbl*

```
Comparison.tbl |> knitr::kable(caption = "Comparative analysis of PC and PCF structures")
```

7.3 Plotting Comparison.tbl

Plotting *Comparison.tbl* using *plotMultiItemCatSharesData()*

```
plotMultiItemCatSharesData(Comparison.tbl)
```



---

8 Appendix

8.1 A Real-Factory Narrative: The Slot-Car Assembly Plant

Prompt/MS-Copilot: Give a narrative description of the real factory case for producing slot-cars that gives a compact overview concering cost and GHG emission accounting.

Answer/MS-Copilot: In a mid-sized manufacturing town, a factory produces a popular line of compact slot-cars. The product is simple enough to be made at scale, yet complex enough to involve multiple materials, machining steps, and assembly operations. To meet both cost and sustainability goals, the factory introduced an **activity-based cost & GHG accounting system**. Every material, activity, and equipment categorey is assigned:

- a **unit cost** (EUR/kg, EUR/h, EUR/kWh)
- a **GHG emission factor** (kgCO2e/kg, kgCO2e/h, kgCO2e/kWh)
- a **routing definition** describing how much labor, machine time, and energy each production step consumes.

This data feeds into a product-level breakdown that reveals:

- **direct costs** (materials)
- **indirect costs** (labor + equipment)
- **Scope 1 emissions** (direct combustion)
- **Scope 2 emissions** (electricity)
- **Scope 3 emissions** (materials + equipment embodied emissions).

With this transparency, engineers can pinpoint improvement opportunities - lighter aluminum tubes, lower-energy welding settings, or more efficient painting cycles - while managers can evaluate the financial impact of design changes or supplier choices.

In short, the factory is a realistic example of why modern manufacturing increasingly relies on **activity-based cost & GHG accounting system**: it enables better decisions, supports regulatory compliance, and strengthens competitiveness in a market where customers care about both price and environmental performance.

8.2 From BOM and ROUTING To MIS: Explaining Details

Question/MS-Copilot: Explain which data are provided by the ERP system, why is there a need for a unified Item master (ITEM.tbl) and how is the unified item master needed for building the MIS graph engine for providing the digital twin of the production system, the recursive engine for calculating the products cost and GHG emissions and the breakdown engine for delivering detailed reports concerning the different cost and GHG scope categories.

Answer/MS-Copilot: Below is a **clear, compact, and conceptually precise explanation** of:

- **what data the ERP system provides**,
- **why a unified Item Master (ITEM.tbl) is necessary**, and
- **how this unified Item Master enables**
  - the **MIS graph engine**,

  - the **recursive cost & GHG engine**, and
  - the **breakdown engine**
    in the flashlight production digital twin.

I'll keep it structured and slide-ready, but still rich enough for academic or technical documentation.

8.2.1 What Data the ERP System Provides

In the flashlight factory, the ERP system supplies **three essential data domains**:

(A) BOM Data (Bill of Materials)
This includes:

- which **materials** and **semi-finished items** are required
- in what **quantities**
- for producing which **target item**

This is the **material-input knowledge**.

(B) ROUTING Data (Process Plan)
This includes:

- which **activity** produces each item
- required **labor time**, **machine time**, **energy use**
- which **equipment** is used
- process-specific **cost** and **emission factors**

This is the **transformation-knowledge**.

(C) Master Data (Materials, Activities, Equipment)
This includes:

- material prices and emission factors
- activity cost rates (labor, overhead)
- equipment cost rates and embodied emissions
- electricity related emission factors (Scope 2)
- direct combustion emissions factors (Scope 1)

This is the **valuation knowledge**.

8.2.2 Why a Unified Item Master (ITEM.tbl) Is Needed

ERP systems typically store BOM and ROUTING in **separate tables**, often with:

- different ID systems,
- different naming conventions,
- different levels of granularity,
- and no guaranteed one-to-one mapping.

This creates three problems:

(A) BOM and ROUTING refer to items differently
BOM may refer to:

- "CoverRed"
- "Underbody"
- "Chassis"

Routing may refer to:

- "Item 4711"
- "Item 4711-A"
- "Assembly A"

Without a unified ID, the system cannot know that:

the BOM input "Chassis" is the same entity that the ROUTING says is produced by assembly.

(B) Items appear in multiple roles
The same item may be:

- a **material** in one context,
- a **semi-finished item** in another,
- a **final product** in a third.

Only a unified Item Master can assign:

- a **single unique ID**,
- a **consistent category**,
- a **consistent set of attributes**,
  to each item.

(C) The MIS graph requires a single ontology
The MIS graph is a **network of hybrid nodes**.
Each hybrid node must contain:

- **item knowledge** (from BOM + master data)
- **activity knowledge** (from routing + master data)

This is impossible unless both BOM and ROUTING refer to the **same item IDs**.

Thus, the unified Item Master is the **semantic backbone** of the entire digital twin.

8.2.3 How the Unified Item Master Enables the MIS Graph Engine

The MIS graph engine builds a **directed transformation graph** where each node is a **hybrid production entity**:

- **v.item** = the produced object
- **v.activity** = the transformation that produces it

To construct this graph, the engine must:

1. **Match BOM inputs** to
2. **The activity that produces the target item**
3. **Using the same item ID**

Without ITEM.tbl, this matching is impossible.

The unified Item Master enables:

- consistent node creation
- consistent edge creation
- consistent attribute assignment
- consistent traversal logic

It is the **ontology** that makes the graph computable.

8.2.4 How the Unified Item Master Enables the Recursive Engine

The recursive engine computes:

- **Cost(v)**
- **GHG(v)**

for every hybrid node **v**, by walking from the leaf nodes upstream through the MIS graph.

This requires:

- consistent item IDs
- consistent activity IDs
- consistent quantities
- consistent cost/emission attributes

If BOM and ROUTING refer to items differently, recursion breaks.

The unified Item Master ensures:

- every upstream dependency is correctly identified
- every material and activity contribution is aggregated
- Scope 1, 2, and 3 emissions are correctly assigned
- equipment and energy contributions are not lost

Thus, ITEM.tbl is the **key enabler** of recursive cost and GHG propagation.

8.2.5 How the Unified Item Master Enables the Accounting Recursion Engine

The accounting engine produces:

- direct material cost
- indirect activity cost
- indirect equipment cost as well as
- Scope 1 / Scope 2 / Scope 3 emissions

To do this, it must:

- trace each contribution back to the correct item
- trace each activity back to the correct node
- aggregate partial costs and GHG emissions by category

This requires:

- a **single item ID** for grouping
- a **single activity ID** for grouping
- consistent mapping between BOM and routing

Without ITEM.tbl, cost and GHG accounting would be:

- incomplete,
- inconsistent,
- or impossible.

8.2.6 In One Sentence

The ERP system provides the raw material, process, and valuation data, but only the unified Item Master (ITEM.tbl) makes it possible to integrate BOM and ROUTING into a coherent MIS graph, enabling the digital twin, the recursive accounting engine, and the cost analysis engine to function correctly and consistently.

8.3 Manual Building of Input Data from ERP System

```r
# Building input data by hand

MAT.erp <- tribble(
  ~matID, ~matName,                 ~pM,   ~uMEL,
  1,      "CoverRed",               29.00,   0.15,
  2,      "CoverBlue",              28.00,   0.10,
  3,      "Motor",                  10.00,   0.20,
  4,      "MotorMountingBracket",    0.12,   0.0006,
  5,      "CScrew_M2x6",             0.02,   0.05,
  6,      "UnderBody",               1.44,   0.0072,
  7,      "PowerSystem",             3.26,   0.06,
  8,      "WheelAssembly",          13.40,   0.766,
  9,      "WheelAssemblyDriven",    18.20,   1.10)

SEMI.erp <- tribble(
  ~spID, ~spName,
  50,    "DriveUnit",
  51,    "Chassis")

FINAL.erp <- tribble(
  ~fpID, ~fpName,
  100,    "SlotCarRed",
  101,    "SlotCarBlue")

BOM.erp <- tribble(
  ~bomID, ~parCat, ~parID, ~childCat, ~childID, ~rM,
  1,      "semi",   50,     "mat",      3,        1,
  2,      "semi",   50,     "mat",      4,        1,
  3,      "semi",   50,     "mat",      5,        1,
  4,      "semi",   51,     "mat",      6,        1,
  5,      "semi",   51,     "mat",      7,        1,
  6,      "semi",   51,     "mat",      8,        1,
  7,      "semi",   51,     "mat",      9,        1,
  8,      "final",  100,     "mat",      1,        1,
  9,      "final",  100,     "mat",      5,        1,
 10,      "final",  100,     "semi",    50,        1,
 11,      "final",  100,     "semi",    51,        1,
 12,      "final",  101,     "mat",      2,        1,
 13,      "final",  101,     "mat",      5,        1,
 14,      "final",  101,     "semi",    50,        1,
 15,      "final",  101,     "semi",    51,        1)

ACTY.erp <- tribble(
  ~actyID, ~actyName,             ~pA,
  1,       "DriveUnitAssembly",   2,
  2,       "ChassisAssembly",     2,
  3,       "SlotCarAssembly",     2)

ENERGY.erp <- tribble(
  ~enyID, ~enyName,      ~emE,  ~scopeCat,
  1,      "Electricity", 0.069, 2,
  2,      "Gas",         0.069, 1)
```

```r
EQIP.erp <- tribble(
  ~eqipID, ~eqipName,            ~qE, ~cuEEL,     ~enyID,
  1,       "AssemblerElectric",  0.3,  0.00312,   1,
  2,       "AssemblerGas",       0.3,  0.00312,   2)

ROUTING.erp <- tribble(
  ~routeID, ~semFinCat, ~semFinID, ~actyID, ~rE,    ~eqipID,    ~powE,
  1,        "semi",      50,        1,       7.5,    2,          0.05,
  2,        "semi",      51,        2,      51.0,    1,          0.05,
  3,        "final",     100,       3,       1.5,    1,          0.05,
  4,        "final",     101,       3,       1.5,    1,          0.05)
```